

COMPLETE TEXTBOOK NOTES ON TREES

Trees in Data Structures & Algorithms

TREE	
ROOT	
■	■
LEFT SUBTREE	RIGHT SUBTREE

Professional textbook-style study edition

Prepared from the conversation content, with the original instructional material retained in sequence.

Source Conversation

User request — preserved as part of the conversation record.

Create a professional textbook-style PDF from the entire conversation above on Trees, preserving every single word exactly as it appears in the conversation.

Instructions:

1. Preserve Content Exactly

- Include 100% of the conversation content.
- Do not remove, summarize, paraphrase, rewrite, shorten, or modify any text.
- Preserve all questions, answers, code snippets, examples, explanations, formulas, tables, and notes exactly as given.
- Maintain the original sequence and context of the conversation.
- Ensure not a single word is omitted.

2. Convert into Textbook Format

- Create a professional title page.
- Generate a complete table of contents.
- Organize the content into logical chapters, sections, and subsections based on topics discussed.
- Use chapter numbering and hierarchical headings.
- Add page numbers, headers, and footers.

3. Academic Presentation

- Format content like a university-level textbook.
- Highlight important concepts, definitions, theorems, formulas, and takeaways using callout boxes.
- Present code examples in properly formatted code blocks.
- Format equations and mathematical expressions clearly.
- Use professional typography, spacing, and layout standards.

4. Visual Enhancements

- Create relevant diagrams, flowcharts, mind maps, concept maps, and illustrations where applicable.
- Add topic-wise visual summaries.
- Include chapter divider pages.
- Use clean and readable educational design elements.

5. Learning Support

- Add chapter summaries at the end of each chapter.
- Generate key points and revision notes without altering the original conversation text.
- Include glossary and index sections if applicable.

6. PDF Quality Requirements

- Produce a print-ready, high-resolution PDF.
- Ensure proper margins, alignment, and readability.

- Keep formatting consistent throughout the document.
- Ensure all images, charts, diagrams, and tables are properly positioned.

7. Final Verification

- Verify that every word from the original conversation has been included.
- Verify that no content has been lost during restructuring.
- Verify that all code blocks and examples remain unchanged.
- Ensure the final PDF is complete, accurate, and professionally formatted.

Output:

Generate a polished textbook-style PDF titled:

"Complete Textbook Notes on Trees"

The final PDF must contain the entire conversation verbatim while presenting it in a structured, professional textbook format suitable for long-term study, revision, teaching, and reference.

Table of Contents

The numbered sections below follow the original sequence of the Trees explanation.

1. What is a Tree?
2. Why Do We Need Trees?
3. Basic Tree Terminology
4. Root
5. Edge
6. Parent
7. Child
8. Siblings
9. Leaf Node
10. Internal Node
11. Degree of a Node
12. Degree of a Tree
13. Path
14. Ancestor
15. Descendant
16. Subtree
17. Depth
18. Height
19. Depth vs Height
20. Level
21. Types of Trees
22. General Tree
23. Binary Tree
24. Binary Tree Node
25. Full Binary Tree
26. Complete Binary Tree
27. Perfect Binary Tree
28. Perfect Binary Tree Formula
29. Maximum Nodes at a Given Level
30. Maximum Nodes in a Binary Tree
31. Minimum Height of a Binary Tree
32. Degenerate Tree

- 33. Balanced Binary Tree
- 34. Tree Traversal
- 35. Example Tree
- 36. Preorder Traversal
- 37. Preorder Algorithm
- 38. Inorder Traversal
- 39. Inorder Algorithm
- 40. Postorder Traversal
- 41. Postorder Algorithm
- 42. Level Order Traversal
- 43. Level Order Algorithm
- 44. DFS vs BFS
- 45. Traversal Summary
- 46. Important Trick for Traversals
- 47. Iterative Preorder Traversal
- 48. Iterative Inorder
- 49. Iterative Postorder
- 50. Time Complexity of Traversals
- 51. Binary Search Tree
- 52. BST Search
- 53. BST Search Algorithm
- 54. BST Insertion
- 55. BST Inorder Property
- 56. BST Complexity
- 57. BST Minimum
- 58. BST Maximum
- 59. Lowest Common Ancestor
- 60. LCA in a BST
- 61. Tree Diameter
- 62. Height of a Tree
- 63. Maximum Depth
- 64. Counting Nodes
- 65. Sum of All Nodes
- 66. Search in a Normal Binary Tree

67. Tree Recursion Pattern
68. Why Recursion Works So Well for Trees
69. The Three Questions for Tree Recursion
70. Example: Maximum Value in a Tree
71. Example: Check if Two Trees Are Identical
72. Example: Invert a Binary Tree
73. Tree Problems Often Have Two Approaches
74. Top-Down Tree Recursion
75. Bottom-Up Tree Recursion
76. Tree Dynamic Programming
77. Heap
78. Heap vs BST
79. Heap Array Representation
80. Trie
81. Trie Complexity
82. AVL Tree
83. Red-Black Tree
84. Tree Rotations
85. Tree Representation
86. Tree Given as Edges
87. Tree Traversal When Input Is an Edge List
88. Tree vs Graph
89. Tree Properties
90. Tree Traversal When Input Is an Edge List
91. Common Tree Interview Patterns
92. Important Tree Problems for Interviews
93. Complexity Cheat Sheet
94. Tree Complexity Is Usually About Height
95. The Most Important Mental Model
96. Tree Problem-Solving Framework
97. The Core Tree Template
98. Tree Learning Roadmap
99. What You Should Memorize
100. Final Mental Picture

Original Trees Explanation — Verbatim Study Content

The following section preserves the complete Trees explanation in its original sequence and wording; formatting is applied only to presentation.

Trees in DSA — Complete Detailed Explanation

Trees are one of the most important **non-linear data structures** in DSA. They appear extensively in coding interviews, databases, file systems, compilers, operating systems, and many real-world applications.

If you understand trees properly, you will have a strong foundation for:

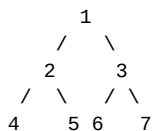
- Binary Trees
 - Binary Search Trees (BST)
 - Heaps
 - AVL Trees
 - Red-Black Trees
 - Tries
 - Segment Trees
 - Expression Trees
 - Decision Trees
 - Lowest Common Ancestor (LCA)
 - Tree Dynamic Programming
 - Graph traversal concepts
-

1. What is a Tree?

A **tree** is a hierarchical data structure consisting of **nodes connected by edges**.

Unlike arrays and linked lists, trees are generally **non-linear**.

For example:



Here:

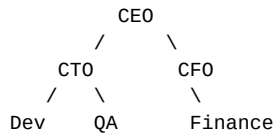
- 1 is the topmost node.
- 2 and 3 are connected to 1.
- 4 and 5 are connected to 2.

- 6 and 7 are connected to 3.

The tree represents a hierarchy.

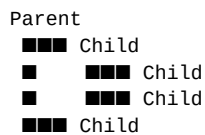
2. Why Do We Need Trees?

Consider a company hierarchy:



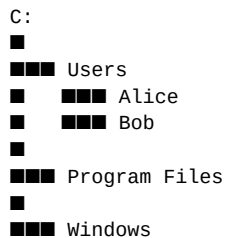
Representing this using an array would be inconvenient.

A tree naturally represents:

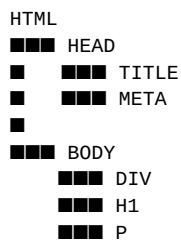


Other examples:

File system



HTML DOM



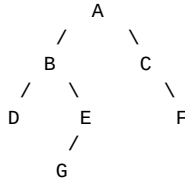
Database indexes

Trees such as **B-Trees** and **B+ Trees** are widely used for indexing.

3. Basic Tree Terminology

Understanding tree terminology is extremely important for interviews.

Consider:



3.1 Node

A **node** is an individual element of a tree.

A

A, B, C, etc. are nodes.

A node typically contains:

```
data
links to children
```

For a binary tree:

```
class Node:
    data
    left
    right
```

4. Root

The **root** is the topmost node of a tree.



A is the root.

A tree has exactly **one root**.

5. Edge

An **edge** is the connection between two nodes.



The connection:

$A \rightarrow B$

is an edge.

If a tree contains N nodes, it always contains:

$N - 1$ edges

assuming it is a connected tree.

6. Parent

A node directly above another node is its **parent**.



A is the parent of:

B
C

7. Child

A node directly below another node is its **child**.



B and C are children of A.

8. Siblings

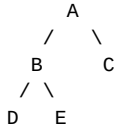
Nodes having the same parent are called **siblings**.



B and C are siblings.

9. Leaf Node

A node with **no children** is called a leaf node.



Leaf nodes:

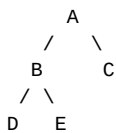
C
D
E

They have no children.

10. Internal Node

A node having at least one child is an **internal node**.

In:



Internal nodes:

A
B

11. Degree of a Node

The **degree of a node** is the number of children it has.

Example:



Degree of A:

3

Degree of:

$$B = 0$$

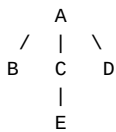
$$C = 0$$

$$D = 0$$

12. Degree of a Tree

The degree of a tree is the **maximum degree of any node**.

Example:



Maximum number of children of any node = 3.

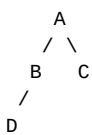
Therefore:

$$\text{Degree of tree} = 3$$

13. Path

A path is a sequence of nodes connected through edges.

For:



Path from A to D:

$$A \rightarrow B \rightarrow D$$

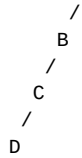
Number of edges:

$$2$$

14. Ancestor

If a node appears above another node in its path, it is an ancestor.

A



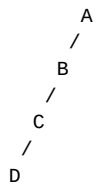
For D:

Ancestors = C, B, A

15. Descendant

The reverse concept is descendant.

For:



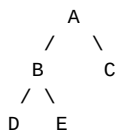
Descendants of A:

B
C
D

16. Subtree

A node together with all its descendants forms a **subtree**.

Consider:



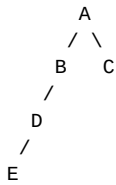
Subtree rooted at B:



17. Depth

The **depth of a node** is the number of edges from the root to that node.

Example:



Depth:

A = 0
B = 1
C = 1
D = 2
E = 3

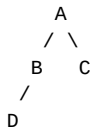
Therefore:

`depth(node) = number of edges from root to node`

18. Height

The height of a node is the number of edges on the **longest path from that node to a leaf**.

Example:



For D:

`height(D) = 0`

For B:

`B → D`

Therefore:

`height(B) = 1`

For A:

`A → B → D`

Therefore:

`height(A) = 2`

The height of the tree is the height of its root.

19. Depth vs Height

This is a very common interview question.

Depth

Measures:

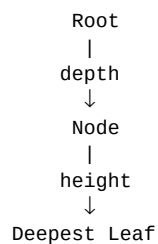
Root → Node

Height

Measures:

Node → Deepest Leaf

Visual:



So:

Depth = distance from root

Height = distance to deepest leaf

20. Level

There are two conventions commonly used.

Convention 1

Root is at level 0:

Level 0: A
Level 1: B C
Level 2: D E F

Convention 2

Root is at level 1.

In coding interviews, always check which convention is being used.

A common convention is:

`level = depth + 1`

when root is level 1.

21. Types of Trees

There are many types of trees.

Important ones for DSA:

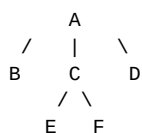
```
Tree
■
■■■ General Tree
■
■■■ Binary Tree
■
■ ■■■ Full Binary Tree
■ ■■■ Complete Binary Tree
■ ■■■ Perfect Binary Tree
■ ■■■ Balanced Binary Tree
■ ■■■ Degenerate Tree
■
■■■ Binary Search Tree
■
■■■ AVL Tree
■
■■■ Red-Black Tree
■
■■■ Heap
■
■■■ Trie
■
■■■ B-Tree / B+ Tree
```

Let's understand them.

22. General Tree

A general tree allows a node to have any number of children.

Example:



A has 3 children.

There is no fixed limit.

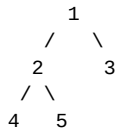
23. Binary Tree

A **binary tree** is a tree in which every node has at most **two children**.

Those children are called:

```
left child
right child
```

Example:



A node can have:

```
0 children
1 child
2 children
```

but never more than 2.

24. Binary Tree Node

Typical Python implementation:

```
class TreeNode:
    def __init__(self, value):
        self.val = value
        self.left = None
        self.right = None
```

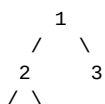
Creating a tree:

```
root = TreeNode(1)

root.left = TreeNode(2)
root.right = TreeNode(3)

root.left.left = TreeNode(4)
root.left.right = TreeNode(5)
```

This creates:



4 5

25. Full Binary Tree

A **full binary tree** is a binary tree where every node has either:

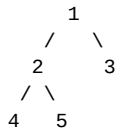
0 children

or:

2 children

Never exactly one child.

Example:



Valid full binary tree.

But:



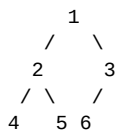
is not full because node 1 has exactly one child.

26. Complete Binary Tree

A complete binary tree has:

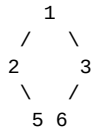
1. Every level completely filled except possibly the last.
2. The last level is filled from **left to right**.

Example:



This is complete.

But:



is not complete.

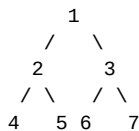
This concept is extremely important for **heaps**.

27. Perfect Binary Tree

A perfect binary tree has:

- Every internal node has exactly 2 children.
- All leaf nodes are at the same level.

Example:



28. Perfect Binary Tree Formula

If height is h , number of nodes is:

$$N = 2^{(h+1)} - 1$$

For:

$$h = 0$$

$$\begin{aligned} N &= 2^1 - 1 \\ &= 1 \end{aligned}$$

For:

$$h = 1$$

$$\begin{aligned} N &= 2^2 - 1 \\ &= 3 \end{aligned}$$

For:

$$h = 2$$

$$\begin{aligned} N &= 2^3 - 1 \\ &= 7 \end{aligned}$$

29. Maximum Nodes at a Given Level

For a binary tree, maximum nodes at level L are:

$$2^L$$

assuming root is level 0.

Example:

Level 0 $\rightarrow$ 1 node
Level 1 $\rightarrow$ 2 nodes
Level 2 $\rightarrow$ 4 nodes
Level 3 $\rightarrow$ 8 nodes

30. Maximum Nodes in a Binary Tree

For height h :

$$\text{Maximum nodes} = 2^{(h+1)} - 1$$

31. Minimum Height of a Binary Tree

For N nodes, the minimum possible height is approximately:

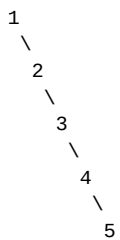
$$\text{ceil}(\log_2(N + 1)) - 1$$

This occurs when the tree is as balanced as possible.

32. Degenerate Tree

A degenerate tree behaves like a linked list.

Example:



or:

```
1
|
2
|
3
|
4
|
5
```

Height:

$N - 1$

This is important because many tree operations become:

$O(N)$

instead of:

$O(\log N)$

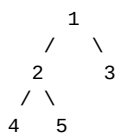
33. Balanced Binary Tree

A binary tree is generally considered height-balanced when the heights of the left and right subtrees differ by at most a small constant, commonly 1.

For every node:

$|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

Example:



The tree is balanced.

Balanced trees are important because they keep height close to:

$O(\log N)$

34. Tree Traversal

Traversal means:

Visiting every node of a tree in a specific order.

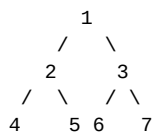
The most important binary tree traversals are:

1. Preorder
2. Inorder
3. Postorder
4. Level Order

These are extremely important for interviews.

35. Example Tree

We will use this tree:



36. Preorder Traversal

Order:

Root → Left → Right

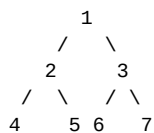
Mnemonic:

NLR

where:

N = Node
L = Left
R = Right

For our tree:



Visit:

1
2
4
5
3
6
7

Therefore:

```
Preorder = [1, 2, 4, 5, 3, 6, 7]
```

37. Preorder Algorithm

```
def preorder(root):  
    if root is None:  
        return  
  
    print(root.val)  
  
    preorder(root.left)  
    preorder(root.right)
```

The sequence is:

```
visit root  
↓  
traverse left  
↓  
traverse right
```

38. Inorder Traversal

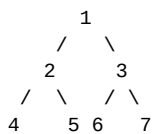
Order:

Left → Root → Right

Mnemonic:

LNR

For:



Visit:

```
4  
2  
5  
1  
6  
3  
7
```

Therefore:

```
Inorder = [4, 2, 5, 1, 6, 3, 7]
```

39. Inorder Algorithm

```
def inorder(root):  
    if root is None:  
        return  
  
    inorder(root.left)  
  
    print(root.val)  
  
    inorder(root.right)
```

40. Postorder Traversal

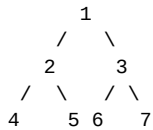
Order:

Left → Right → Root

Mnemonic:

LRN

For our tree:



Visit:

4
5
2
6
7
3
1

Therefore:

Postorder = [4, 5, 2, 6, 7, 3, 1]

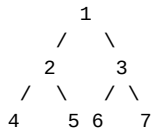
41. Postorder Algorithm

```
def postorder(root):  
    if root is None:  
        return  
  
    postorder(root.left)  
    postorder(root.right)  
  
    print(root.val)
```

42. Level Order Traversal

Level order visits nodes level by level.

Example:



Traversal:

```
Level 0 → 1
Level 1 → 2, 3
Level 2 → 4, 5, 6, 7
```

Therefore:

```
[1, 2, 3, 4, 5, 6, 7]
```

Level order traversal uses a:

Queue

43. Level Order Algorithm

```
from collections import deque

def level_order(root):
    if root is None:
        return []

    queue = deque([root])
    result = []

    while queue:
        node = queue.popleft()

        result.append(node.val)

        if node.left:
            queue.append(node.left)

        if node.right:
            queue.append(node.right)

    return result
```

44. DFS vs BFS

Tree traversal can be categorized into:

DFS



■■■ Preorder

■■■ Inorder

■■■ Postorder

BFS



■■■ Level Order

DFS

Uses:

Recursion

or:

Stack

BFS

Uses:

Queue

45. Traversal Summary

Traversal	Order	Typical Data Structure
Preorder	Root → Left → Right	Stack / Recursion
Inorder	Left → Root → Right	Stack / Recursion
Postorder	Left → Right → Root	Stack / Recursion
Level Order	Level by level	Queue

Memorize:

Pre → NLR

In → LNR

Post → LRN

46. Important Trick for Traversals

Look at where the root is processed.

Preorder

ROOT comes FIRST

Root → Left → Right

Inorder

ROOT comes IN BETWEEN

Left → Root → Right

Postorder

ROOT comes LAST

Left → Right → Root

This simple idea makes the three traversals much easier to remember.

47. Iterative Preorder Traversal

Recursive traversal is easy, but interviewers may ask for an iterative solution.

Use a stack.

```
def preorder(root):
    if root is None:
        return []

    stack = [root]
    result = []

    while stack:
        node = stack.pop()

        result.append(node.val)

        if node.right:
            stack.append(node.right)

        if node.left:
            stack.append(node.left)

    return result
```

Why push right first?

Because stack is LIFO:

Last In → First Out

We want:

Left before Right

So:

```
push right
push left
```

48. Iterative Inorder

```
def inorder(root):
    stack = []
    result = []

    current = root

    while stack or current:

        while current:
            stack.append(current)
            current = current.left

        current = stack.pop()

        result.append(current.val)

        current = current.right

    return result
```

This pattern is extremely important.

Remember:

```
Go left as far as possible
↓
Process node
↓
Go right
```

49. Iterative Postorder

One approach uses two stacks.

```
def postorder(root):
    if root is None:
        return []

    stack1 = [root]
    stack2 = []
    result = []

    while stack1:
        node = stack1.pop()

        stack2.append(node)

        if node.left:
            stack1.append(node.left)

        if node.right:
            stack1.append(node.right)

    while stack2:
        result.append(stack2.pop().val)

    return result
```

50. Time Complexity of Traversals

Every node is visited exactly once.

Therefore:

$$\text{Time} = O(N)$$

where N is the number of nodes.

Space:

Recursive DFS

$$O(H)$$

where H is tree height.

Worst case:

$$H = N$$

Therefore:

$$\text{Worst-case space} = O(N)$$

For a balanced tree:

$$H = O(\log N)$$

so recursion stack is:

$$O(\log N)$$

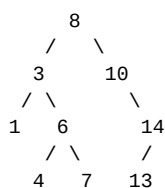
51. Binary Search Tree

A **Binary Search Tree (BST)** is a binary tree with an ordering property.

For every node:

$$\text{Left subtree values} < \text{Node value} < \text{Right subtree values}$$

Example:



For node 8:

Left values < 8
Right values > 8

52. BST Search

Suppose we want to search for:

7

Start:

8

Since:

$7 < 8$

go left.

3

Since:

$7 > 3$

go right.

6

Since:

$7 > 6$

go right.

7

Found.

We don't need to inspect every node.

53. BST Search Algorithm

```
def search(root, target):  
    if root is None:  
        return None  
  
    if root.val == target:  
        return root
```

```
if target < root.val:
    return search(root.left, target)

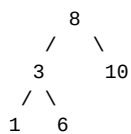
return search(root.right, target)
```

54. BST Insertion

Suppose we insert:

5

into:



Compare:

$5 < 8$

go left.

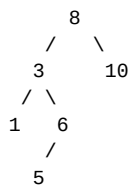
$5 > 3$

go right.

$5 < 6$

go left.

Insert:



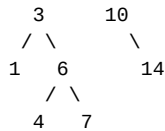
55. BST Inorder Property

This is one of the most important tree facts:

Inorder traversal of a valid BST produces sorted order.

Example:





Inorder:

1 3 4 6 7 8 10 14

Sorted!

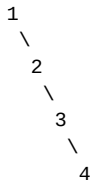
This property is frequently used in interview problems.

56. BST Complexity

If the BST is balanced:

Search = $O(\log N)$
Insert = $O(\log N)$
Delete = $O(\log N)$

But if the tree becomes skewed:



then:

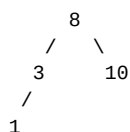
Search = $O(N)$
Insert = $O(N)$
Delete = $O(N)$

Therefore:

BST performance depends heavily on tree height.

57. BST Minimum

The minimum value is the leftmost node.



Minimum:

1

Algorithm:

```
def find_min(root):  
    current = root  
  
    while current.left:  
        current = current.left  
  
    return current
```

58. BST Maximum

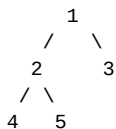
The maximum value is the rightmost node.

```
def find_max(root):  
    current = root  
  
    while current.right:  
        current = current.right  
  
    return current
```

59. Lowest Common Ancestor

The **Lowest Common Ancestor (LCA)** of two nodes is the deepest node that is an ancestor of both.

Example:



LCA of:

4 and 5

is:

2

LCA of:

4 and 3

is:

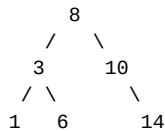
1

LCA is a very common interview topic.

60. LCA in a BST

For a BST, the ordering property makes LCA easier.

Suppose:



Find LCA of:

1 and 6

At 8:

$1 < 8$
 $6 < 8$

Both are on the left.

Move to 3.

At 3:

$1 < 3$
 $6 > 3$

They split here.

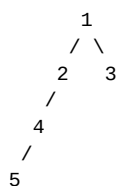
Therefore:

LCA = 3

61. Tree Diameter

The **diameter** of a tree is the length of the longest path between any two nodes.

Example:



Longest path:

5 → 4 → 2 → 1 → 3

Number of edges:

4

Therefore:

Diameter = 4

Important:

Some problems define diameter in terms of **nodes**, while others define it in terms of **edges**.

Always check the definition.

62. Height of a Tree

Height can be calculated recursively.

```
def height(root):
    if root is None:
        return -1

    return 1 + max(
        height(root.left),
        height(root.right)
    )
```

Using this convention:

```
empty tree = -1
leaf = 0
```

Another common convention uses:

```
empty tree = 0
leaf = 1
```

Both are valid depending on the problem.

63. Maximum Depth

LeetCode-style problems often define maximum depth as number of nodes on the longest root-to-leaf path.

Then:

```
def max_depth(root):
    if root is None:
        return 0

    return 1 + max(
```

```
        max_depth(root.left),
        max_depth(root.right)
    )
```

For:

```
1
 \
  2
   \
    3
```

maximum depth:

3

64. Counting Nodes

Simple recursive approach:

```
def count_nodes(root):
    if root is None:
        return 0

    return (
        1
        + count_nodes(root.left)
        + count_nodes(root.right)
    )
```

Time:

$O(N)$

65. Sum of All Nodes

```
def tree_sum(root):
    if root is None:
        return 0

    return (
        root.val
        + tree_sum(root.left)
        + tree_sum(root.right)
    )
```

66. Search in a Normal Binary Tree

Unlike a BST, a normal binary tree has no ordering property.

Therefore, to search for a value, you may need to inspect every node.

```
def contains(root, target):
    if root is None:
        return False

    if root.val == target:
        return True

    return (
        contains(root.left, target)
        or contains(root.right, target)
    )
```

Worst case:

$O(N)$

67. Tree Recursion Pattern

Many tree problems follow the same structure:

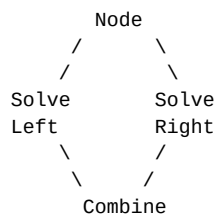
```
def solve(root):
    if root is None:
        return base_case

    left_result = solve(root.left)
    right_result = solve(root.right)

    return combine(
        root,
        left_result,
        right_result
    )
```

This is one of the most important patterns to master.

Think:



68. Why Recursion Works So Well for Trees

A tree is naturally recursive.

A tree consists of:

```
Root
+
Left subtree
+
```

Right subtree

And each subtree is itself a tree.

Therefore:

```
Tree
↓
Root + Tree + Tree
```

This recursive structure maps directly to recursive code.

69. The Three Questions for Tree Recursion

When solving a tree problem, ask:

Question 1

What should I return for:

```
root == None
```

Question 2

What information do I need from:

```
left subtree
right subtree
```

Question 3

How do I combine them at the current node?

This framework solves a huge number of tree problems.

70. Example: Maximum Value in a Tree

```
def maximum(root):
    if root is None:
        return float("-inf")

    left_max = maximum(root.left)
    right_max = maximum(root.right)

    return max(root.val, left_max, right_max)
```

At each node:

```
maximum =
max(
    current value,
```

```
    left subtree maximum,  
    right subtree maximum  
)
```

71. Example: Check if Two Trees Are Identical

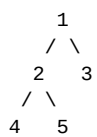
Two trees are identical if:

1. Their values at corresponding nodes are equal.
2. Their left subtrees are identical.
3. Their right subtrees are identical.

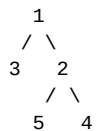
```
def same_tree(p, q):  
    if p is None and q is None:  
        return True  
  
    if p is None or q is None:  
        return False  
  
    if p.val != q.val:  
        return False  
  
    return (  
        same_tree(p.left, q.left)  
        and same_tree(p.right, q.right)  
    )
```

72. Example: Invert a Binary Tree

Given:



Invert it:



Algorithm:

```
def invert_tree(root):  
    if root is None:  
        return None  
  
    root.left, root.right = root.right, root.left  
  
    invert_tree(root.left)
```

```
invert_tree(root.right)

return root
```

73. Tree Problems Often Have Two Approaches

A tree problem can frequently be solved using:

Top-down approach

or:

Bottom-up approach

74. Top-Down Tree Recursion

Information flows:

```
Parent
↓
Child
↓
Grandchild
```

Example:

Root-to-leaf path sum

You pass information down.

```
def dfs(node, current_sum):

    if node is None:
        return

    current_sum += node.val

    dfs(node.left, current_sum)
    dfs(node.right, current_sum)
```

75. Bottom-Up Tree Recursion

Information flows:

```
Leaf
↓
Parent
↓
Root
```

For example:

```
height
diameter
balance
maximum subtree value
```

Example:

```
def height(root):
    if root is None:
        return 0

    left = height(root.left)
    right = height(root.right)

    return 1 + max(left, right)
```

The children calculate their answers first, and the parent uses them.

76. Tree Dynamic Programming

Tree DP is an advanced but extremely important concept.

Example:

```
    10
   /  \
  5    20
```

Suppose we want some property of the entire tree.

We can calculate:

```
left subtree answer
right subtree answer
current node contribution
```

Then:

```
answer(node)
=
combine(
    answer(left),
    answer(right),
    node
)
```

This pattern appears in many advanced interview problems.

77. Heap

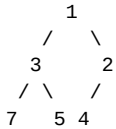
A heap is a specialized complete binary tree.

Two major types:

Min Heap
Max Heap

Min Heap

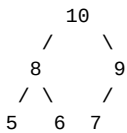
Parent is smaller than or equal to children.



Minimum element is always at root.

Max Heap

Parent is greater than or equal to children.



Maximum element is at root.

78. Heap vs BST

This distinction is important.

BST

Optimized around:

Searching ordered values

Property:

Left < Root < Right

Heap

Optimized around:

Minimum/Maximum element

Property:

Parent $\leq$ Children

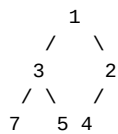
for min heap.

A heap does **not** maintain complete sorted order.

79. Heap Array Representation

Because a heap is a complete binary tree, it can be efficiently stored in an array.

For:



Array:

[1, 3, 2, 7, 5, 4]

For **0-based indexing**:

$\text{Parent}(i) = (i - 1) // 2$

$\text{Left}(i) = 2*i + 1$

$\text{Right}(i) = 2*i + 2$

These formulas are extremely important.

80. Trie

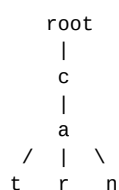
A **Trie** is a tree used primarily for storing strings.

Example:

Words:

cat
car
can

Trie:



Tries are useful for:

- Autocomplete
 - Dictionary lookup
 - Prefix matching
 - Spell checking
 - Word search
-

81. Trie Complexity

If the word length is L :

Insertion:

$O(L)$

Search:

$O(L)$

Prefix search:

$O(L)$

This can be much better than scanning every stored word when prefix operations are required.

82. AVL Tree

An AVL tree is a **self-balancing Binary Search Tree**.

For every node:

$|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$

This difference is called the:

Balance Factor

Formula:

Balance Factor =
 $\text{height}(\text{left subtree})$
 $-$
 $\text{height}(\text{right subtree})$

Valid values:

-1

0
+1

If the balance factor becomes:

-2

or:

+2

the tree needs rebalancing.

AVL trees use rotations.

83. Red-Black Tree

A Red-Black Tree is another self-balancing BST.

Each node has a color:

Red
Black

It follows several structural rules to maintain approximate balance.

Used in implementations of ordered data structures such as:

C++ `std::map`
C++ `std::set`
Java `TreeMap`
Java `TreeSet`

The key idea is:

Self-balancing BST

with operations generally:

$O(\log N)$

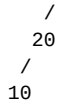
84. Tree Rotations

Rotations are used to maintain balance in BST variants.

Example of a right rotation:

Before:

30



After:



The ordering property remains valid.

Rotations are central to:

AVL Trees
Red-Black Trees

85. Tree Representation

There are several ways to represent trees.

Representation 1: Linked nodes

```
class TreeNode:
    def __init__(self, value):
        self.val = value
        self.left = None
        self.right = None
```

Most common for binary trees.

Representation 2: Array

Especially useful for:

Heap
Complete binary tree

Representation 3: Adjacency list

For general trees:

```
tree = {
    1: [2, 3],
    2: [4, 5],
    3: [6],
    4: [],
    5: [],
    6: []
}
```

This representation is also very common when tree problems are given as **edges**.

86. Tree Given as Edges

Sometimes the input is:

```
edges = [  
    [0, 1],  
    [0, 2],  
    [1, 3],  
    [1, 4]  
]
```

This represents:

```
  0  
 / \  
1   2  
/ \  
3  4
```

You can construct an adjacency list:

```
from collections import defaultdict  
  
graph = defaultdict(list)  
  
for u, v in edges:  
    graph[u].append(v)  
    graph[v].append(u)
```

Because the edges are undirected.

87. Tree Traversal When Input Is an Edge List

Suppose:

```
0 -- 1  
|    |  
2    3
```

Adjacency list:

```
graph = {  
    0: [1, 2],  
    1: [0, 3],  
    2: [0],  
    3: [1]  
}
```

DFS:

```
def dfs(node, parent):  
    print(node)  
    for neighbor in graph[node]:
```

```
if neighbor == parent:
    continue

dfs(neighbor, node)
```

Why do we track parent?

Because the tree is represented as an undirected graph.

For example:

$0 \leftrightarrow 1$

When at 1, we don't want to immediately go back to 0.

88. Tree vs Graph

This is an important conceptual distinction.

A tree is actually a special type of graph.

A tree has:

Connected
+
Acyclic

properties.

For N nodes:

Edges = $N - 1$

and there is exactly one simple path between any two nodes.

89. Tree Properties

A tree with N nodes has:

$N - 1$ edges

A tree is:

Connected

A tree is:

Acyclic

Between any two nodes:

Exactly one simple path

Removing any edge:

disconnects the tree

Adding any new edge:

creates a cycle

These are very useful graph-theory properties.

90. Tree Traversal When Input Is an Edge List

Suppose:

```
0 -- 1
|    |
2    3
```

Adjacency list:

```
graph = {
    0: [1, 2],
    1: [0, 3],
    2: [0],
    3: [1]
}
```

DFS:

```
def dfs(node, parent):
    print(node)
    for neighbor in graph[node]:
        if neighbor == parent:
            continue
        dfs(neighbor, node)
```

Why do we track parent?

Because the tree is represented as an undirected graph.

For example:

```
0 ↔ 1
```

When at 1, we don't want to immediately go back to 0.

91. Common Tree Interview Patterns

You should learn trees through **patterns**, not by memorizing individual problems.

Important patterns include:

Pattern 1 — DFS

```
Preorder
Inorder
Postorder
```

Pattern 2 — BFS

```
Level order
```

Pattern 3 — Tree recursion

```
Solve left
Solve right
Combine
```

Pattern 4 — Height

```
1 + max(left, right)
```

Pattern 5 — Diameter

Usually based on subtree heights.

Pattern 6 — Path problems

Examples:

```
Root-to-leaf path
Maximum path sum
Path sum
All paths
```

Pattern 7 — BST

Use:

```
left < root < right
```

Pattern 8 — LCA

Find the lowest common ancestor.

Pattern 9 — Tree serialization

Convert:

Tree → String

and:

String → Tree

Pattern 10 — Tree DP

Compute information from children and combine at parent.

92. Important Tree Problems for Interviews

A good progression is:

Beginner

1. Maximum depth of binary tree
2. Minimum depth
3. Invert binary tree
4. Same tree
5. Symmetric tree
6. Count nodes
7. Sum of nodes
8. Search in binary tree
9. Preorder traversal
10. Inorder traversal
11. Postorder traversal
12. Level order traversal

Intermediate

13. Validate BST
14. Search in BST
15. Insert into BST

16. Delete node in BST
17. Lowest Common Ancestor
18. Diameter of binary tree
19. Balanced binary tree
20. Path Sum
21. Binary Tree Right Side View
22. Zigzag Level Order Traversal
23. Construct tree from preorder + inorder
24. Construct tree from inorder + postorder

Advanced

25. Maximum Path Sum
26. Serialize and Deserialize Binary Tree
27. Vertical Order Traversal
28. Boundary Traversal
29. Morris Traversal
30. Kth Smallest in BST
31. Recover BST
32. Tree DP
33. Binary Lifting
34. LCA with preprocessing
35. Heavy-Light Decomposition

93. Complexity Cheat Sheet

Operation	Normal Binary Tree	Balanced BST	Skewed BST
Search	$O(N)$	$O(\log N)$	$O(N)$
Insert	$O(N)^*$	$O(\log N)$	$O(N)$
Delete	$O(N)^*$	$O(\log N)$	$O(N)$
Traversal	$O(N)$	$O(N)$	$O(N)$
Height	$O(N)$	$O(\log N)$	$O(N)$

* depends on the exact representation/operation and whether ordering is available.

94. Tree Complexity Is Usually About Height

A very important interview insight:

For many tree algorithms:

$\text{Time} \approx O(\text{height})$

or:

$\text{Time} \approx O(N)$

depending on whether we visit all nodes.

For a balanced tree:

$\text{height} = O(\log N)$

For a skewed tree:

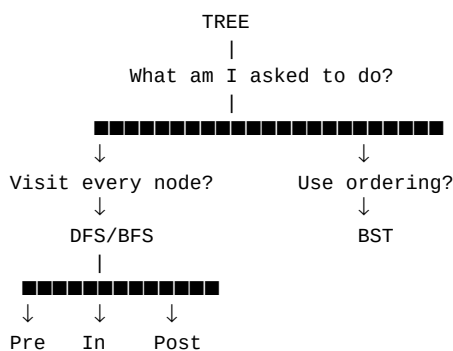
$\text{height} = O(N)$

Therefore:

Balanced tree
↓
small height
↓
faster operations

95. The Most Important Mental Model

When you see a tree problem, first ask:



If the problem asks about levels:

→ BFS / Queue

If it asks about subtree properties:

→ DFS / Recursion

If it is a BST:

→ Exploit ordering

If it asks for longest path:

→ Height / Diameter

If it asks for relationship between two nodes:

→ LCA / Ancestor techniques

96. Tree Problem-Solving Framework

For almost every tree problem, follow this sequence:

Step 1 — Draw the tree

Don't immediately start coding.

Step 2 — Identify the tree type

Ask:

Binary tree?
BST?
Heap?
Trie?
General tree?
Tree represented as edges?

Step 3 — Determine traversal

Ask:

DFS?
BFS?

Step 4 — Decide recursion direction

Ask:

Top-down?
Bottom-up?

Step 5 — Define the function

Clearly define:

What does my recursive function return?

For example:

```
height(node)
```

returns:

Height of the subtree rooted at node.

Step 6 — Define base case

Usually:

```
if root is None:  
    return ...
```

Step 7 — Solve children

```
left = solve(root.left)  
right = solve(root.right)
```

Step 8 — Combine

```
return ...
```

97. The Core Tree Template

This template is worth memorizing:

```
def dfs(root):  
    if root is None:  
        return BASE_CASE  
  
    left = dfs(root.left)  
    right = dfs(root.right)  
  
    return COMBINE(root, left, right)
```

For many tree problems, only these two things change:

BASE_CASE

and:

COMBINE

That's the key to mastering tree recursion.

98. Tree Learning Roadmap

For your DSA interview preparation, I would learn trees in this order:

1. Tree terminology
↓
2. Binary Trees
↓
3. Recursive DFS
↓
4. Preorder
↓
5. Inorder
↓
6. Postorder
↓
7. BFS / Level Order
↓
8. Tree height/depth
↓
9. Tree properties
↓
10. BST
↓
11. BST search/insert/delete
↓
12. LCA
↓
13. Diameter
↓
14. Path problems
↓
15. Tree construction
↓
16. Serialization
↓
17. Advanced Tree DP
↓
18. AVL / Red-Black Trees
↓
19. Trie
↓
20. Segment Tree / Fenwick Tree

99. What You Should Memorize

For interviews, these are the **must-know facts**:

Binary tree

Maximum children = 2

Edges

Tree with N nodes $\rightarrow$ N - 1 edges

Preorder

Root $\rightarrow$ Left $\rightarrow$ Right

Inorder

Left $\rightarrow$ Root $\rightarrow$ Right

Postorder

Left $\rightarrow$ Right $\rightarrow$ Root

BFS

Queue

DFS

Stack / Recursion

BST

Left < Root < Right

BST inorder

Sorted order

Heap

Complete binary tree

Min Heap

Parent $\leq$ Children

Max Heap

Parent $\geq$ Children

Perfect binary tree

Nodes = $2^{(h+1)} - 1$

Heap array

left(i) = $2i + 1$
right(i) = $2i + 2$
parent(i) = $(i-1)//2$

Balanced tree

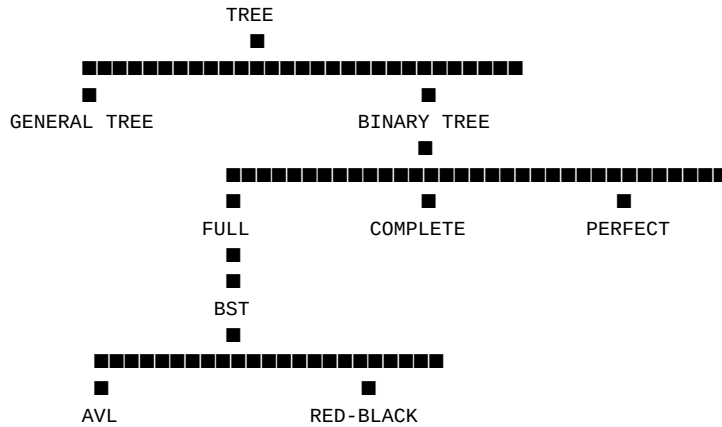
Height $\approx O(\log N)$

Skewed tree

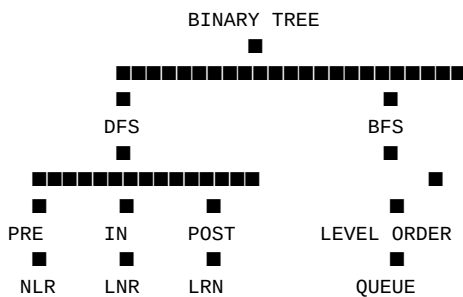
Height $\approx O(N)$

100. Final Mental Picture

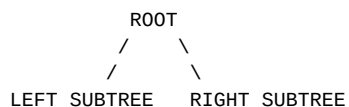
Think of trees as a hierarchy:



And think of traversal as:



The **single most important idea** is that a tree is recursively composed of smaller trees:



Once this becomes intuitive, most binary-tree interview problems reduce to:

```

if root is None:
    return base_case

left = solve(root.left)
right = solve(root.right)

return combine(root, left, right)
  
```

That recursive mental model is the foundation on which you should build **BSTs** → **LCA** → **diameter** → **path problems** → **tree DP** → **advanced tree algorithms**.

Supplementary Study Aids

The following visual and revision aids are additions for study support; they do not replace or alter the preserved conversation content.

TREES IN DSA		
Terminology	Binary Trees	Traversals
BST	Heaps / Tries	Advanced Trees

Study Area	Core Idea
Tree fundamentals	Nodes, edges, root, parent/child, depth, height, subtree.
Binary trees	At most two children; understand full, complete, perfect, balanced and degenerate forms.
Traversal	DFS: preorder, inorder, postorder. BFS: level order.
BST	Ordering enables efficient search, insertion and deletion when height is small.
Advanced structures	Heap, Trie, AVL, Red-Black Tree, tree DP and edge-list trees.

Verification Notes

The source strings used to construct the PDF were retained in memory before formatting. The PDF includes the user request, the full Trees explanation, code examples, formulas, examples, tables, and diagrams in the original sequence. Supplementary study aids are explicitly separated from the source conversation.